

Disease Prediction System Using Machine Learning

Submitted By:

Adithya Menon

TABLE OF CONTENTS

CONTENTS	PAGE NO.
Project Description	3
Dataset Used	4
Environment Setup	5
Data Exploration	6
Data Preprocessing	7
Proposed System	8
Training Pipeline	9-10
Model Evaluation	11 -12
Conclusion	13

1. Project Description

MedAssist AI is a machine learning-based healthcare assistance system designed to analyze user symptoms and predict possible diseases.

The primary objective of the project is to develop a system that can accept a set of symptoms, process them using trained machine learning models, and generate a preliminary disease prediction.

In the previous milestone, the project focused mainly on dataset exploration, preprocessing, machine learning model development, model evaluation, and initial model serialization.

During Milestone 3, the project was further improved by working with a larger disease-symptom dataset containing 240 disease classes and 377 symptom features.

Multiple machine learning algorithms were trained and evaluated. Different team members worked on different models so that their performance could be compared.

The current milestone also focuses on preparing the prediction model for integration with the recommendation module and the backend/frontend application.

The overall project follows the planned disease prediction and healthcare recommendation workflow of MedAssist AI.

2. Dataset Used

For Milestone 3, a larger disease-symptom dataset was used for training and evaluation.

Dataset Characteristics

- Number of records: 151,556
- Total columns: 378
- Disease column: 1
- Symptom features: 377
- Disease classes: 240
- Feature representation: Binary values
- Target variable: Disease

Each symptom is represented as a binary feature:

- 0 → Symptom absent
- 1 → Symptom present

Example:

Disease	Fever	Cough	Headache	...
Panic disorder	1	0	1	...
Panic disorder	0	0	1	...

The larger dataset provides considerably more disease categories and symptom combinations than the dataset used in the previous milestone.

Dataset Structure

151,556 records



377 symptom features



240 disease classes

The dataset is therefore treated as a multi-class classification problem.

3. Environment Setup

The project was developed using Python and machine learning libraries.

Development Environment

- Python
- Jupyter Notebook / Google Colab
- Visual Studio Code
- Git and GitHub

Libraries Used

- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn
- Joblib
- CatBoost

Machine Learning

The project used multiple classification algorithms, including:

- Naive Bayes
- Logistic Regression
- KNN
- Decision Tree
- Random Forest
- Extra Trees

The project architecture also allows additional models such as XGBoost and deep learning models to be incorporated by other team members.

The overall project technology stack is based around Python, Scikit-learn, XGBoost/TensorFlow and related machine-learning libraries.

4. Data Exploration

Before model training, exploratory data analysis was performed on the new dataset.

The following analyses were performed:

- Dataset shape
- Column names
- Data types
- Missing-value analysis
- Duplicate-value checking
- Disease distribution
- Symptom distribution
- Target class distribution

Dataset Shape

The dataset contains:

151,556 rows $\times$ 378 columns

The first column contains the disease name, while the remaining columns represent symptoms.

Missing Values

The dataset was checked for missing values.

The analysis showed that the symptom columns did not contain significant missing values.

Duplicate Analysis

Duplicate records were also checked as part of the data-quality analysis.

Disease Distribution

The distribution of disease classes was visualized using a bar chart.

The analysis showed that the dataset contains multiple samples for each of the 240 disease categories.

5. Data Preprocessing

The following preprocessing steps were performed before model training.

5.1 Feature and Target Separation

The disease column was separated as the target variable.

X = Symptom Features

y = Disease

Therefore:

$X \rightarrow 377$ symptom features

$y \rightarrow 240$ disease classes

5.2 Label Encoding

Since machine learning algorithms require numerical target values, disease names were converted into numerical labels.

For example:

Disease Name

↓

Label Encoding

↓

Numerical Class

The trained label encoder was saved so that predicted numerical classes could later be converted back into disease names.

5.3 Train-Test Split

The dataset was divided into training and testing sets.

Dataset

↓

Train-Test Split

↓

Training Data Testing Data

↓

↓

Model Training Evaluation

5.4 Binary Symptom Representation

The symptoms were already represented using binary values.

Therefore, the symptom features could be directly provided to the machine learning models without requiring additional categorical encoding.

6. Proposed System

The proposed system follows the workflow:

User Symptoms



Symptom Processing



Feature Vector Creation



Machine Learning Model



Disease Prediction



Disease Information



Precautions / Recommendations



Frontend Display

The system is designed to eventually provide not only disease prediction but also healthcare recommendations and preventive information.

The project specification includes disease classification, probability scoring, prediction confidence, treatment recommendations, preventive care suggestions, lifestyle advice, and health reports as major system modules.

7. Training Pipeline

Multiple models were trained independently using the processed dataset.

Training Workflow

Load Dataset



Data Exploration



Data Preprocessing



Feature / Target Separation



Label Encoding



Train-Test Split



Train Multiple Models



Evaluate Models



Compare Performance



Select Candidate Model



Save Model



Prediction Function



Backend Integration

Models Evaluated

The models evaluated as part of the team's machine learning work included:

1. Naive Bayes
2. Logistic Regression
3. KNN
4. Decision Tree
5. Random Forest
6. Extra Trees

Additional boosting and deep-learning approaches are being developed by other members of the team.

8. Model Evaluation

The models were evaluated using the following metrics:

- Accuracy
- Precision
- Recall
- F1 Score

These metrics are important because the system is dealing with a large multi-class disease classification problem.

The project specification also identifies accuracy, precision, recall, F1-score and prediction confidence as important model-performance measures.

8.1 Model Comparison

The following results were obtained from the models evaluated so far.

	Model	Accuracy	Precision	Recall	F1 Score
1	Naive Bayes	0.865994	0.891560	0.865994	0.868893
0	Logistic Regression	0.862464	0.869561	0.862464	0.863417
5	KNN	0.828220	0.832515	0.828220	0.828775
3	Random Forest	0.759006	0.820554	0.759006	0.765231
4	Extra Trees	0.740301	0.813352	0.740301	0.750640
2	Decision Tree	0.137206	0.256354	0.137206	0.167977

Best Result Obtained

Among the models for which complete evaluation metrics are currently available, Naive Bayes achieved the highest accuracy of approximately 86.60%, with:

- Accuracy: 86.60%
- Precision: 89.16%
- Recall: 86.60%
- F1 Score: 86.89%

The teammate's CatBoost implementation achieved:

- Accuracy: 86.30%
- Macro F1: 86.58%

Therefore, both Naive Bayes and CatBoost demonstrated strong performance on the current dataset.

Model Selection

Although Naive Bayes achieved slightly higher accuracy on the evaluation dataset, CatBoost was selected as the final deployment model considering its advantages for complex tabular datasets, ability to capture non-linear relationships between symptoms and diseases, and robustness in handling large-scale multi-class classification problems.

The final CatBoost model achieved:

- Accuracy: 86.30%
- Macro F1 Score: 86.58%

The trained CatBoost model will be integrated with the MedAssistAI backend prediction pipeline for real-time disease prediction.

9. Conclusion

During Milestone 3, the disease prediction system was significantly improved by replacing the previous dataset with a larger and more diverse disease-symptom dataset containing 240 disease classes.

The dataset underwent detailed exploratory data analysis, preprocessing, and class filtering to improve data quality and model reliability. Multiple machine learning and advanced learning approaches were implemented and evaluated, including traditional classifiers, ensemble methods, and deep learning-based approaches.

Among the evaluated models, CatBoost achieved competitive performance with an accuracy of 86.30% and a Macro F1 Score of 86.58%. Considering its ability to handle high-dimensional tabular data, multi-class classification capability, and strong generalization performance, CatBoost was selected as the final prediction model for integration into the MedAssistAI application.

The next phase will focus on integrating the trained CatBoost model with the backend API, connecting the disease information database containing descriptions, precautions, medicines, diet recommendations, lifestyle suggestions, specialist information, and severity levels, and developing the complete user-facing prediction workflow.